



First Semester 2024/2025

Digital Systems ENCS 2340

Verilog HDL ALU Project

Done By : Lara Nidal Daifallah

ID : 1230239

Section : 2

DR : Mohammad Khalil

Table of contents covered in the report:

- **Introduction to Digital Systems**
- **About the ALU** (Arithmetic Logic Unit)
- **Full Adder** (code implementation and waveform simulation result)
- **Logical Operations:**
- **4-Bitwise And** (code implementation and waveform simulation result)
- **4-Bitwise OR** (code implementation and waveform simulation result)
- **Arithmetic Operations:**
- **Addition - Subtraction operations** (code implementation and waveform simulation result)
- **ALU using DataFlow** (code implementation and waveform simulation result)
- **ALU using Behavioral** (code implementation and waveform simulation result)
- **Block Diagram for the ALU**
- **Conclusion**

- **Introduction to Digital Systems**

Digital systems are an important part of today's technology. They use numbers, mainly 0s and 1s, to process, store, and send information. We see them in devices like computers, phones, and even in cars and machines. These systems work quickly and accurately, making them useful in many areas. Learning about digital systems helps us understand how modern devices work and how to improve them. They are a big reason why technology keeps getting better and more advanced.

The language that was used in this project in order to make the digital system is Verilog HDL using Quartus software.

- **About the ALU (Arithmetic Logic Unit)**

The Arithmetic Logic Unit (ALU) is a small but important part of a computer. A 4-bit ALU (our project base) works with numbers that are 4 bits long. It can do four main operations: Addition (to add numbers), Subtraction (to subtract numbers), Bitwise OR (to compare bits and give the oring result), and Bitwise AND (to compare bits and give the anding result). These simple tasks are used in many computer processes and are essential for making calculations and decisions in digital systems.

- Full Adder :

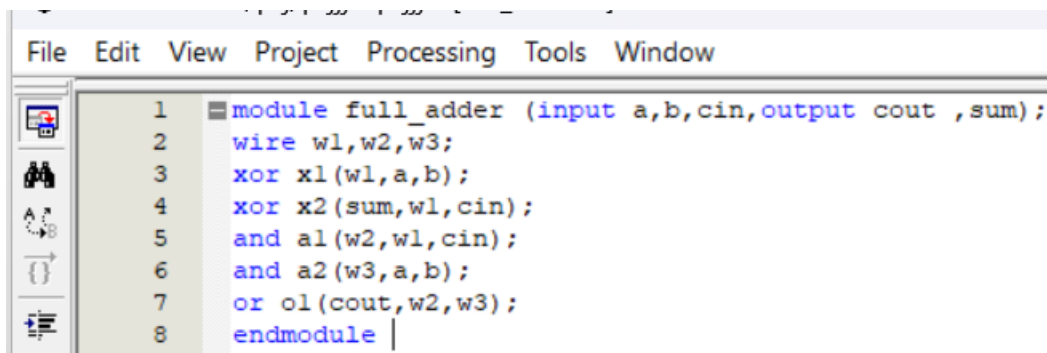
A full adder is a digital circuit that adds three binary inputs: two input bits (a and b) and a carry-in bit (cin) from a previous stage. It produces two outputs: the (sum) and a Carry-out (cout). The logic for the outputs is:

$$\text{sum} = a \oplus b \oplus \text{cin}$$

$$\text{cout} = (a \text{ AND } b) \text{ OR } (\text{cin AND } (a \oplus b))$$

This simple circuit is the building block for creating multi-bit adders which are used in computers and digital systems for binary arithmetic. And it's implemented using gate level modeling

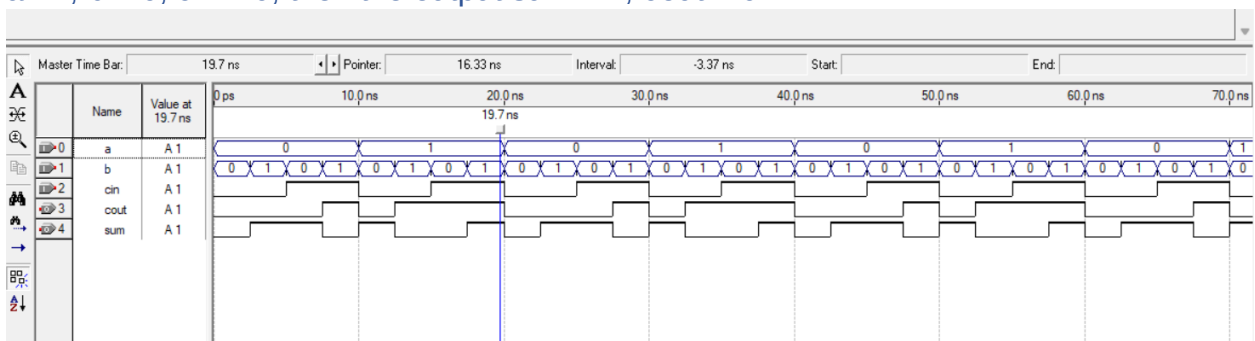
- Code implementation:



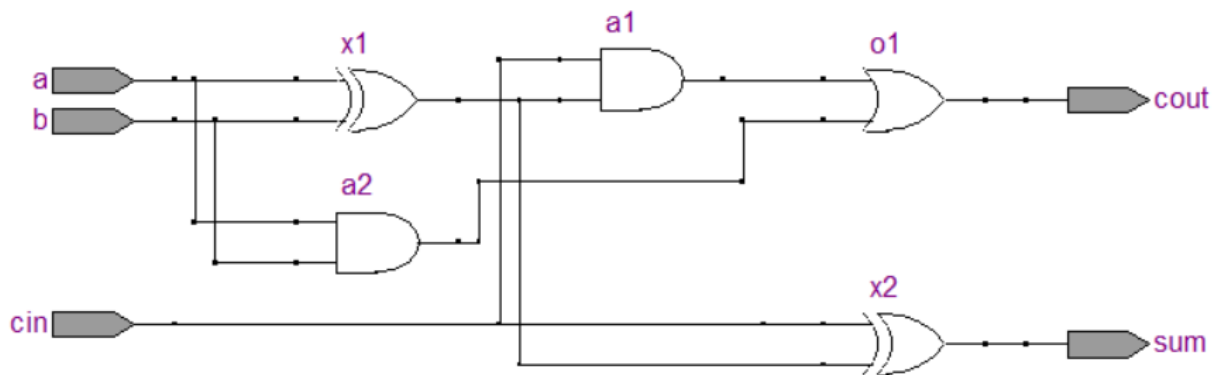
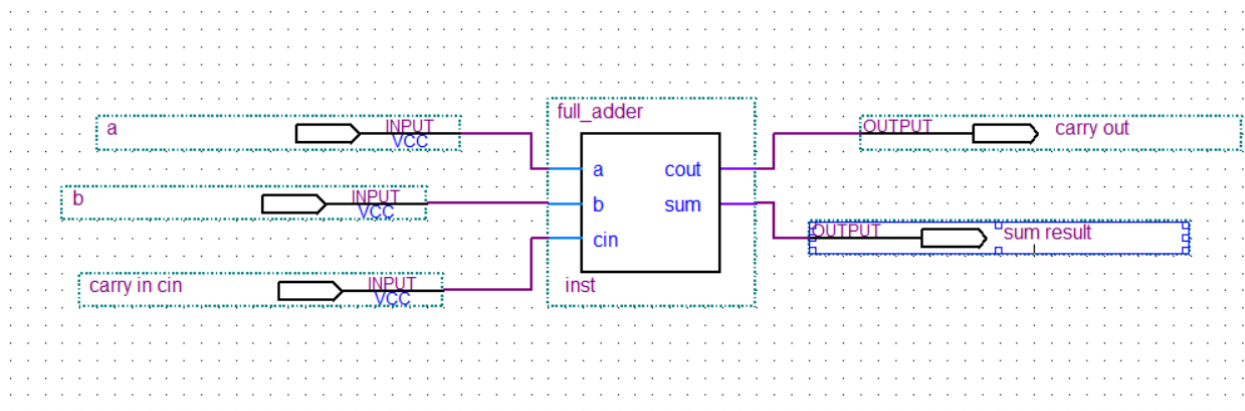
```
File Edit View Project Processing Tools Window
1 module full_adder (input a,b,cin,output cout ,sum);
2   wire w1,w2,w3;
3   xor x1(w1,a,b);
4   xor x2(sum,w1,cin);
5   and a1(w2,w1,cin);
6   and a2(w3,a,b);
7   or o1(cout,w2,w3);
8   endmodule
```

- Waveform : (simulation result):

We use the waveform to analyze the results. As shown the binary input a= 1, b = 0, cin = 0, then the output sum = 1, cout = 0.



- **Block diagram:**



- **Logical Operations:**

- **Bitwise AND :**

It is implemented in the structural modeling (Gates modeling), that we use and gate that it is already exist in the Verilog HDL, and using two 4-bits inputs, and a 4-bits output, we implement the 4-bits using [3:0]. It compares each bit of (a) with the bit with the same index of (b) and puts their anding result in (and_res).

- Code implementation:

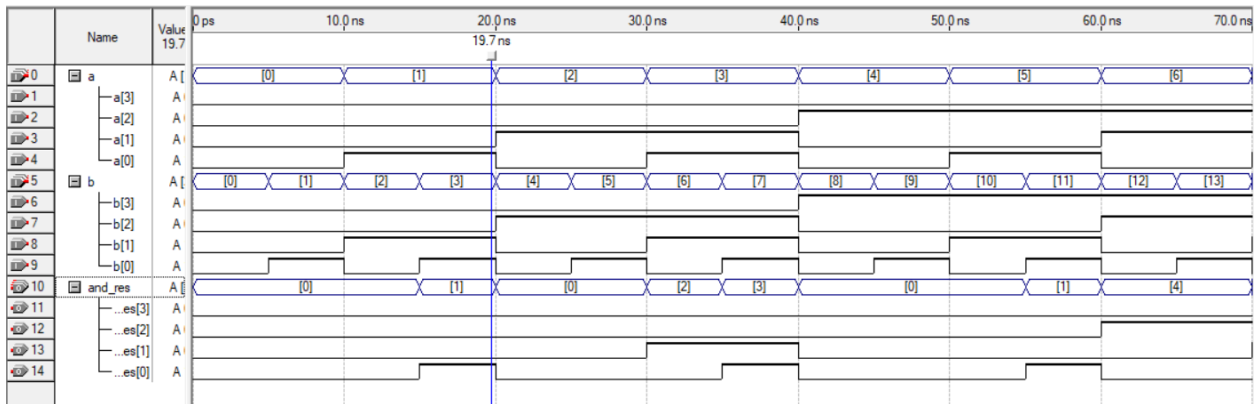
```

1 module bitwise_and(input [3:0] a,b,output [3:0]and_res);
2   and and1 (and_res[0],a[0],b[0]);
3   and and2 (and_res[1],a[1],b[1]);
4   and and3 (and_res[2],a[2],b[2]);
5   and and4 (and_res[3],a[3],b[3]);
6 endmodule

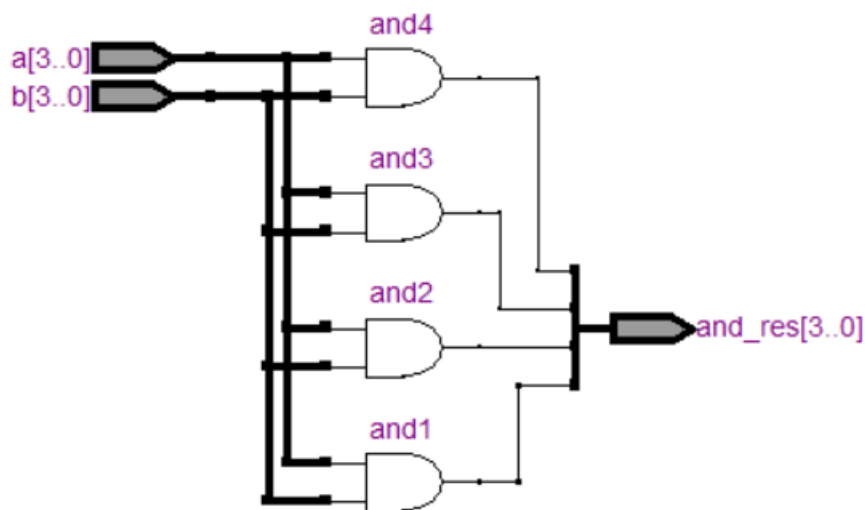
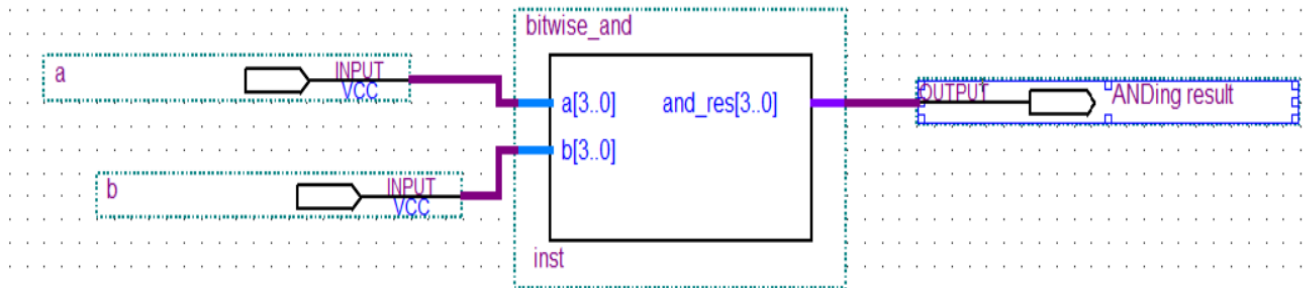
```

- Waveform : (simulation result):

Here for example if we have a=1=0001 and b=3=0011 then their anding result is 0001=1



- **Block diagram:**



- **Bitwise OR :**

It is implemented in the structural modeling (Gates modeling), that we use or gate that it is already exist in the Verilog HDL, and using two 4-bits inputs, and a 4-bits output, we implement the 4-

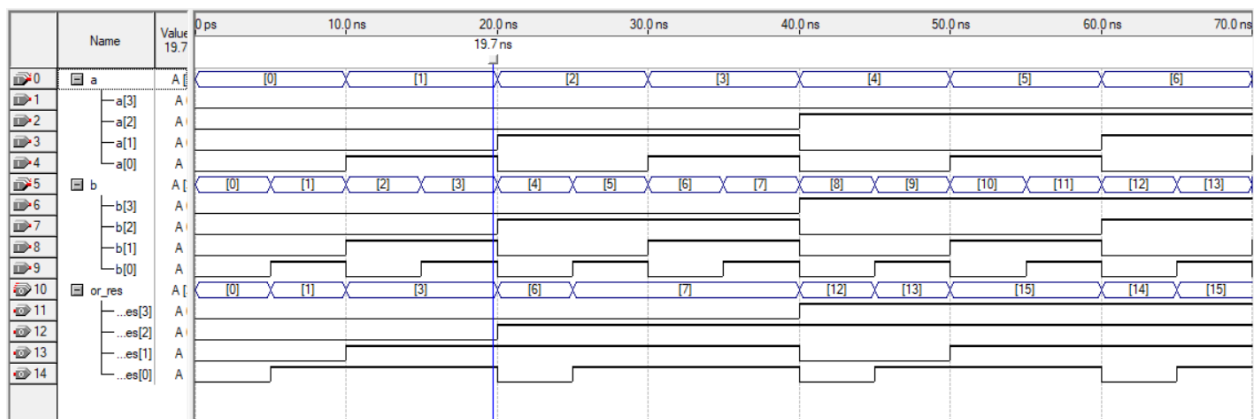
bits using [3:0]. It compares each bit of (a) with the bit with the same index of (b) and puts their oring result in (or_res).

- **Code implementation:**

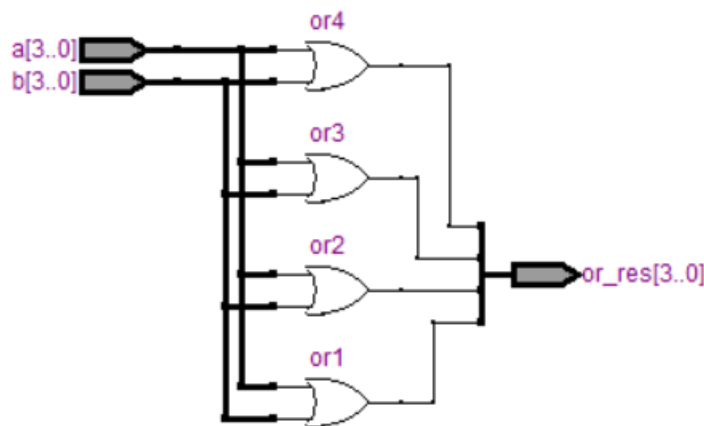
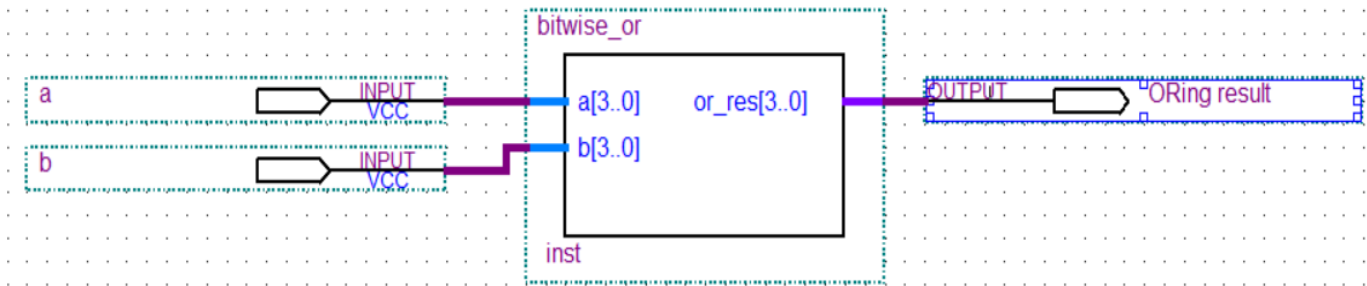
```
File Edit View Project Processing Tools Window
1 module bitwise_or(input [3:0] a,b,output [3:0]or_res);
2   or or1(or_res[0],a[0],b[0]);
3   or or2(or_res[1],a[1],b[1]);
4   or or3(or_res[2],a[2],b[2]);
5   or or4(or_res[3],a[3],b[3]);
6 endmodule
```

- **Waveform : (simulation result):**

Here for example if we have a=1=0001 and b=3=0011 then their oring result is 0011=3



- **Block diagram:**



- **Arithmetic Operations:**

- **Addition and Subtraction Operations :**

- **Addition:**

It is implemented in the structural modeling using

four full adders (one for each bit), we use wires as carry out for each full adder, and we take b as it is :

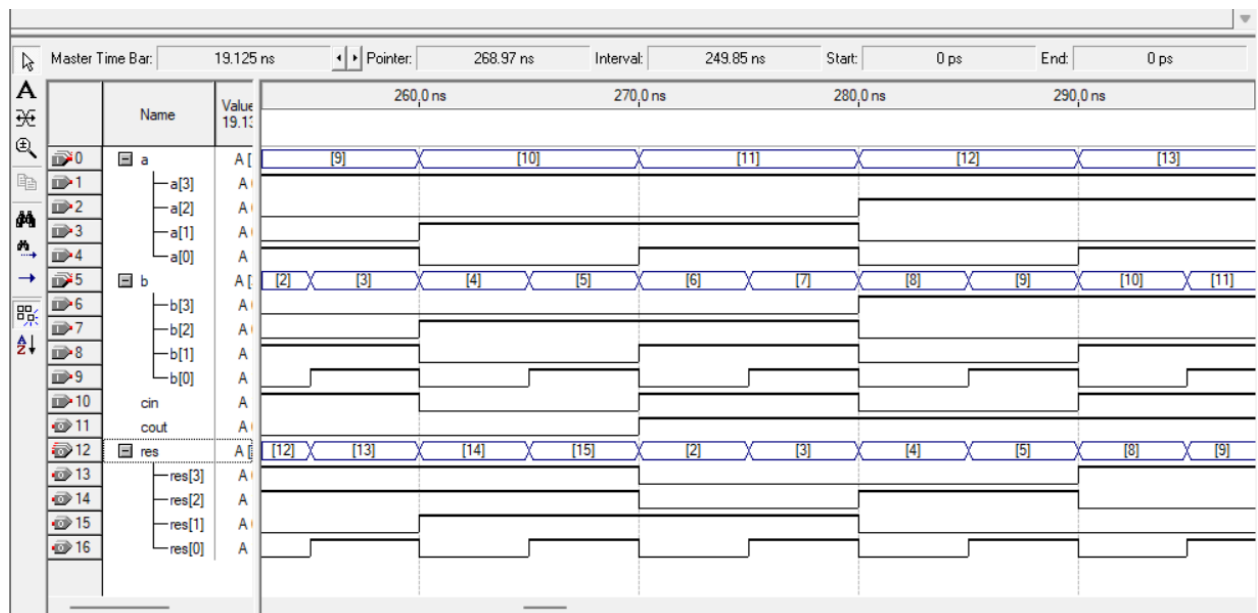
Code implementation:

```
Waveform1.vwf | or_waveform.vwf | full_adder_wav... | datflow_wavef... | Simulator Tool | Simulation Rep... | Compilation R

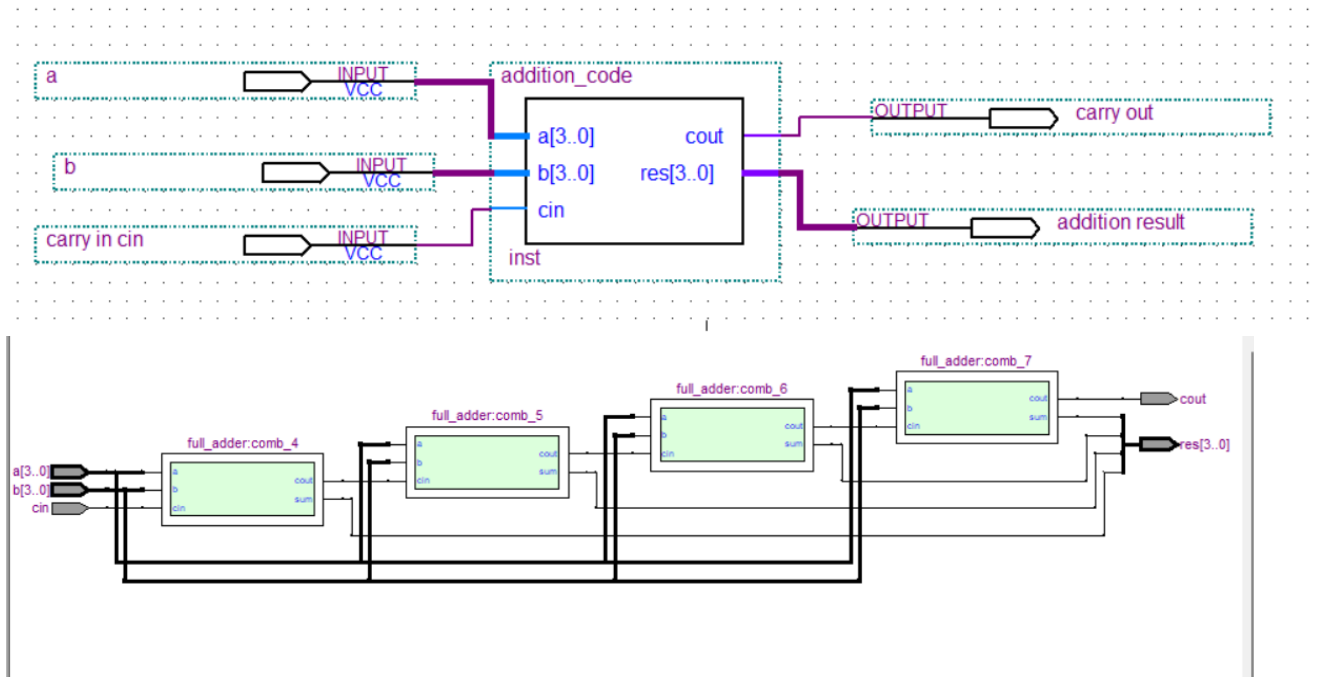
1 module addition_code(input [3:0]a, b, input cin, output cout ,output [3:0] res ) ;
2   wire [2:0]C;
3   full_adder (a[0],b[0],cin, C[0],res[0]);
4   full_adder (a[1],b[1],C[0],C[1],res[1]);
5   full_adder (a[2],b[2],C[1],C[2],res[2]);
6   full_adder (a[3],b[3],C[2],cout,res[3]);
7 endmodule
```

- Waveform : (simulation result):

Here for example the operation is addition , for example we have $a=10=1010$ and $b=4=0100$ then their addition result is $14=1110$ and the carry out is zero since $14 < 16$ (maximum value) and another case $11+6=17$ and $cin=1$ so $a+b=18 > 16$ so $cout=1$ and $res=2$



- **Block diagram:**



- **Subtraction :**

It is implemented in the structural modeling using four full adders (one for each bit), we use wires as carry out for each full adder, and we take b complement because we have subtraction

- Code implementation:

```

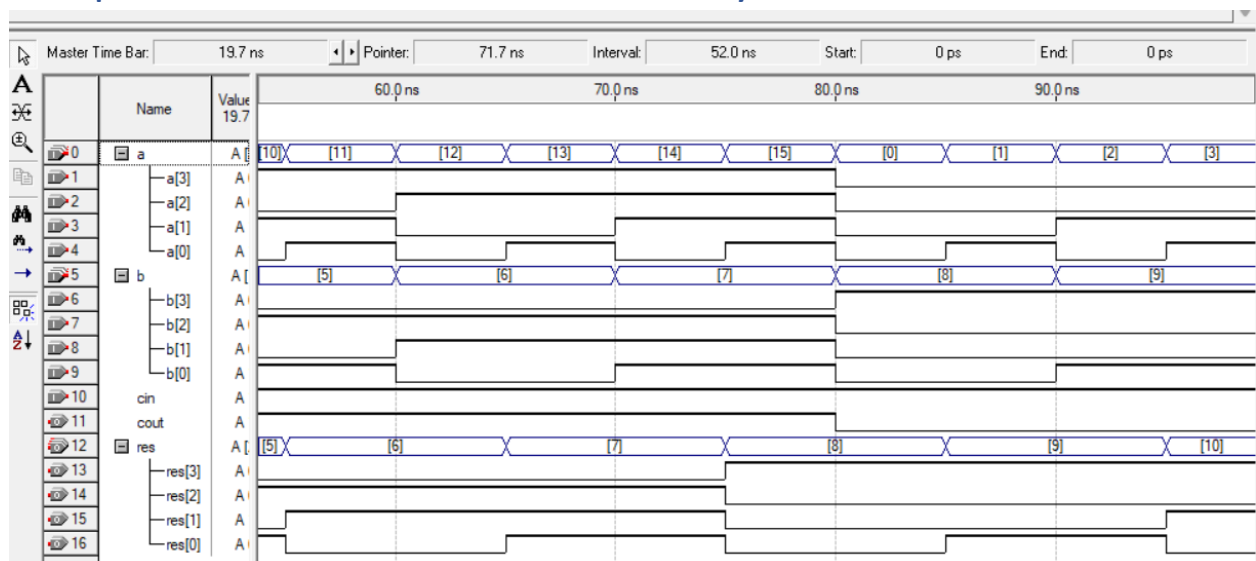
Waveform... | or_wavfor... | full_adder... | datflow_wa... | Simulator T... | Simulation ... | Compilation... | alu_dataflo... | abc a
1  module subtraction_code(input [3:0]a, b, input cin, output cout ,output [3:0] res ) ;
2  wire [2:0]C;// wires to vonnect full adders
3  full_adder (a[0],~b[0],cin, C[0],res[0]);
4  full_adder (a[1],~b[1],C[0],C[1],res[1]);
5  full_adder (a[2],~b[2],C[1],C[2],res[2]);
6  full_adder (a[3],~b[3],C[2],cout,res[3]);
7  //we wrote ~b because we have subtraction operation (and we need b complement)
8  endmodule |

```

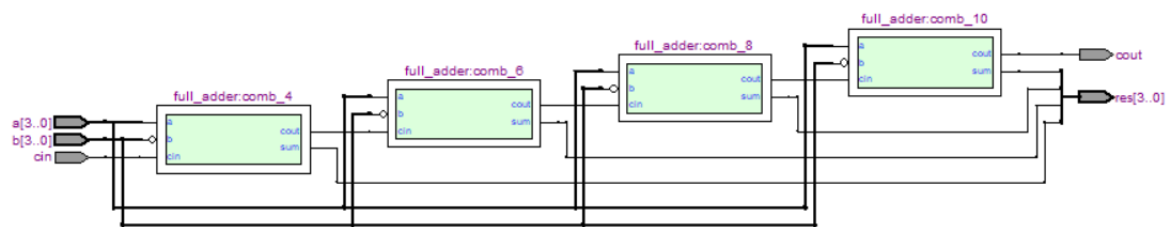
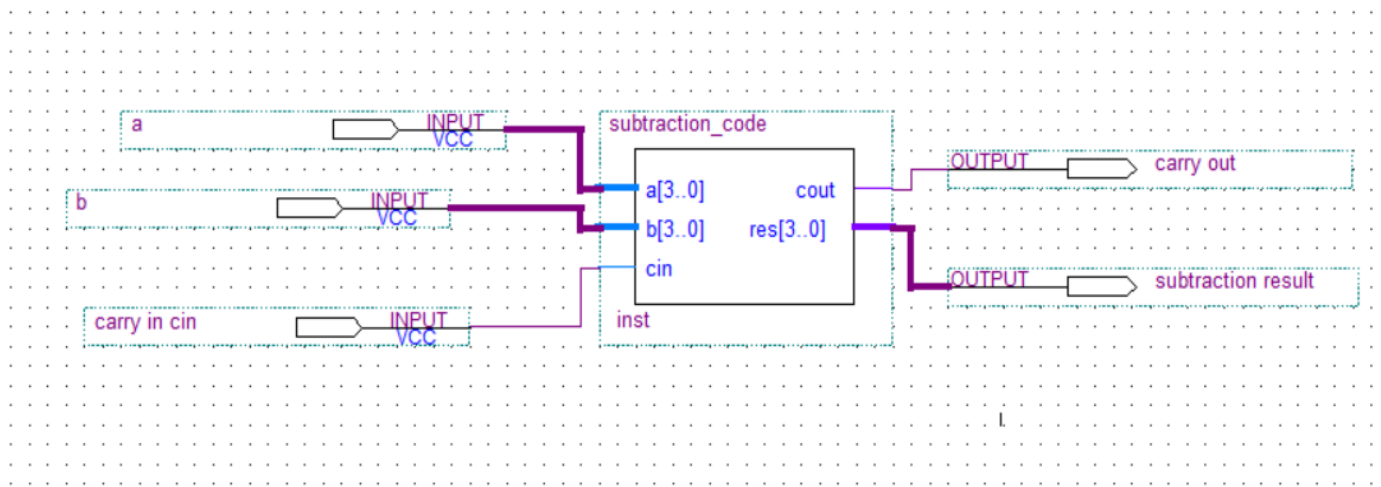
- Waveform : (simulation result):

Here for example the operation is subtraction for example we have $a=14=1110$ and $b=7=0111$ then their subtraction result is $7=0111$ and the carry out is 1 since $a > b$ then there's a carry

While if $a=1=0001$ and $b=8=1000$ then $a-b= -7$ which is the 1st complement of 9 and there is no carry $a < b$



- **Block diagram:**



- **ALU using DataFlow Modeling :**

To connect the previous operations we use dataflow modeling, that we implement 4-wires using the keyword wire, then we call the operations and use the keyword assign to select the operations by the input: 3-bits op code, that it calls the wires which represent the output of operations.

The op code work as a selection if it is equal to 0 then it will operate the addition module with cin = 0 as addition, if op = 1, then it will operate as it is in subtraction module with cin = 1 as subtraction, if op= 2, then the bitwise_and will operate, if op = 3, then the bitwise_and will operate, and if op equals any other number between 4 and 7 it will result a don't care value (Z)

- **Code implementation:**

```

module alu_dataflow(input [3:0]a,b ,input [2:0]op,output cout_add,cout_sub ,output [3:0]result );
wire [3:0] w_OR,w_AND,w_ADD,w_SUB;
bitwise_and (a,b ,w_AND );
bitwise_or (a,b ,w_OR);
addition_code (a,b ,1'b0 ,cout_add ,w_ADD);//addition
subtraction_code (a,b ,1'b1 ,cout_sub ,w_SUB);//subtraction
assign result = (op==3'b000)? w_SUB:
                (op==3'b001)? w_ADD:
                (op==3'b010)? w_OR:
                (op==3'b011)? w_AND: 4'bzzzz;
                //z:(don't care)
                //else don't care
endmodule

```

- **Waveform: (simulation result):**

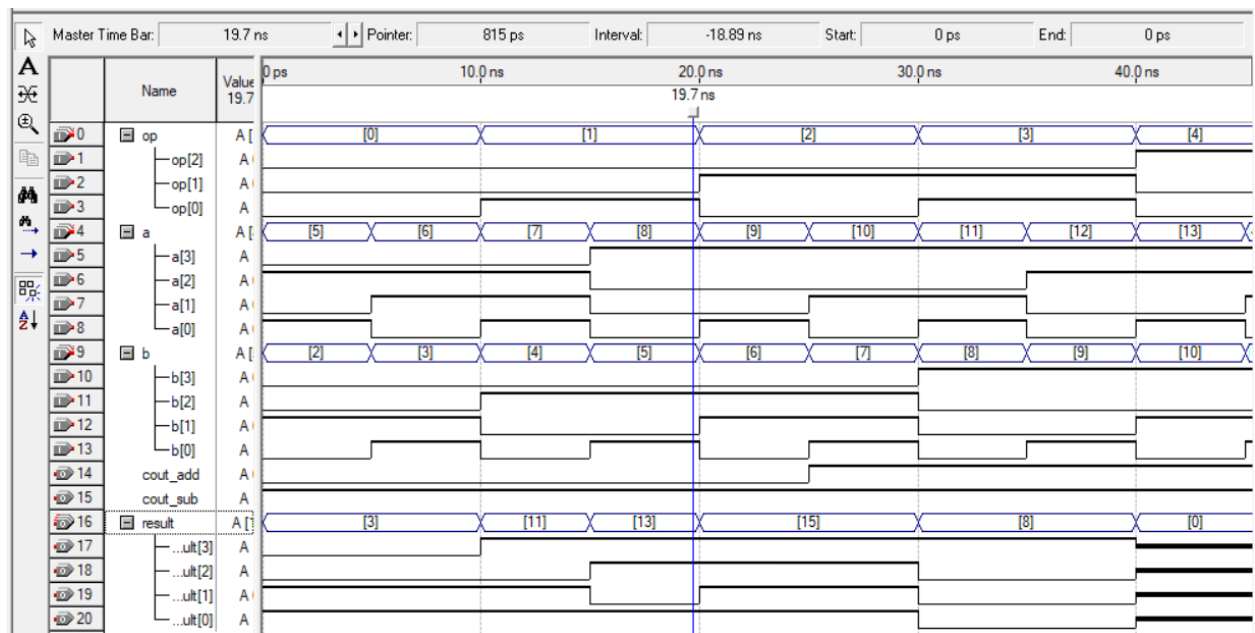
As shown if the input op = 0 then subtraction, a =5, b=2, then the output result =3

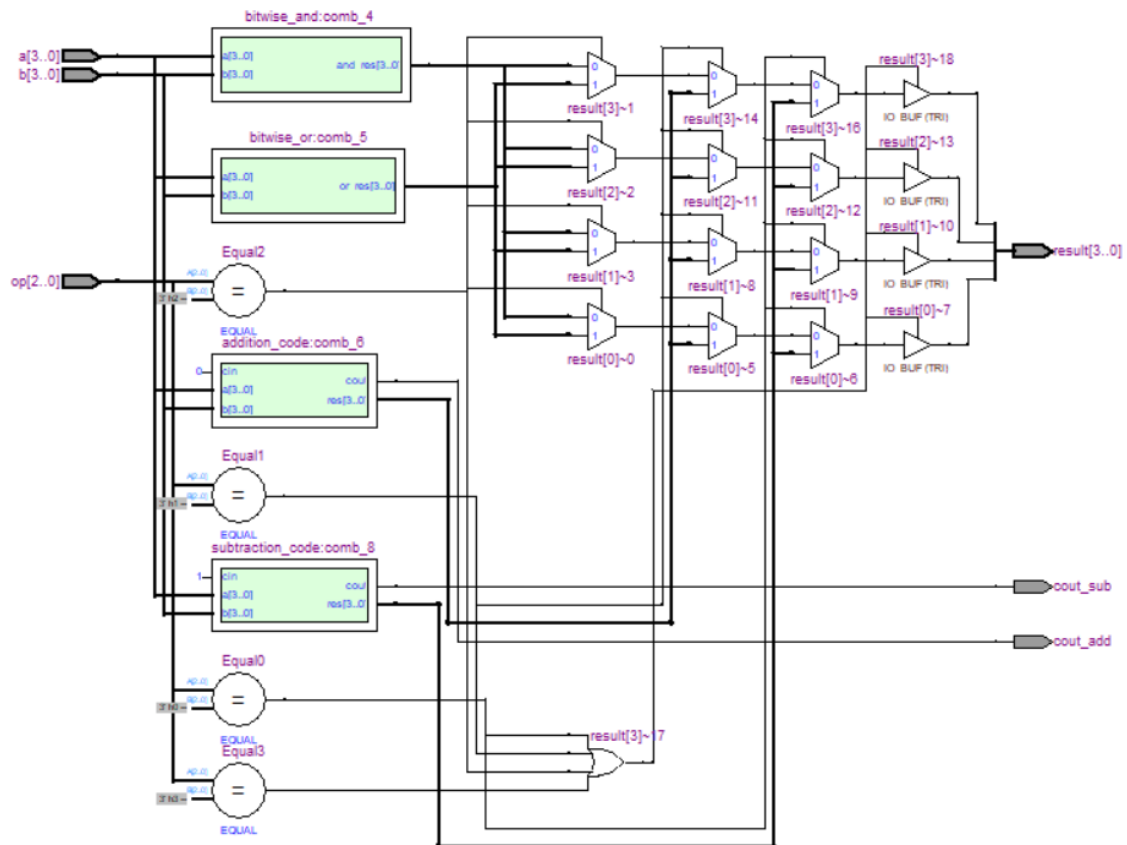
op = 1 then addition, a = 7, b = 4, then the output result =11

op = 2 then or, a= 9, b = 6, then the output result =15.

op = 3 then and, a = 11, b =8, then the output result =8

otherwise it's null





- **ALU using Behavioral Modeling :**

This is another way to connect the previous operations the behavioral modeling, that we implement 4-wires using the keyword wire ,then we call the operations, then we let the op code work as a selection ,we use the keyword always @(*) {* because all the inputs affect the output}, then we use case statement to implement all op code options that it call the wires which in this case they represent the output of the operations .

- Code implementation:

```
1 module alu_behavioral(input [3:0] a,b, input [2:0] op ,output reg [3:0] Cout,output cout_add,cout_sub);
2
3 wire [3:0] wiAnd,wiOr,wiAdd,wiSub;
4 //call the previous operation (instantiation)
5 bitwise_and ( a,b,wiAnd);
6 bitwise_or (a,b,wiOr);
7 addition_code (a,b,1'b0,cout_add,wiAdd); // Addition
8 subtraction_code (a,b,1'b1,cout_sub,wiSub); // Subtraction
9 always @(*) begin // @(*) is @(a,b,cin)
10     case (op)
11         3'b000: Cout = wiSub; // for subtraction
12         3'b001: Cout = wiAdd; // for addition
13         3'b010: Cout = wiOr; // for bitwise OR
14         3'b011: Cout = wiAnd; // for bitwise AND
15         default: Cout = 4'bzzzz; // for don't care when op is greater than 3
16     endcase
17 end
18 endmodule
```

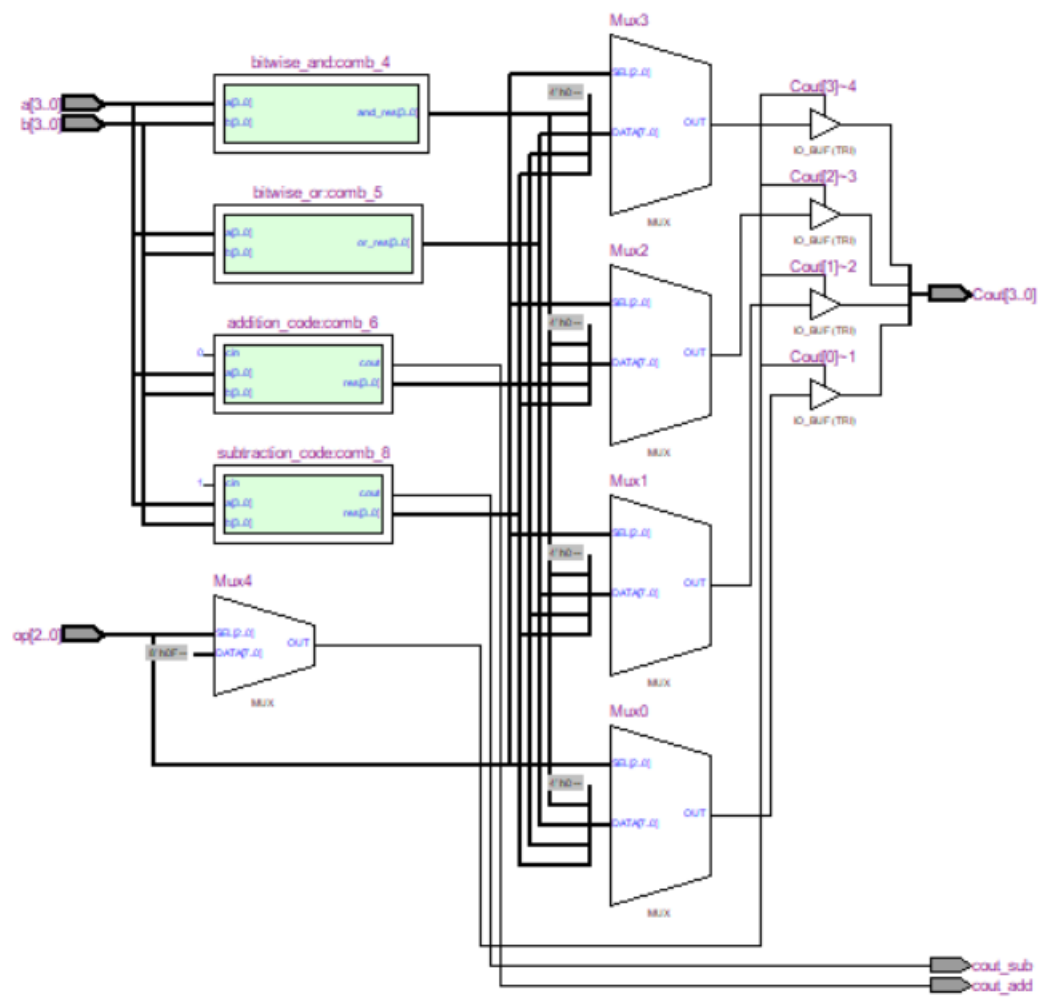
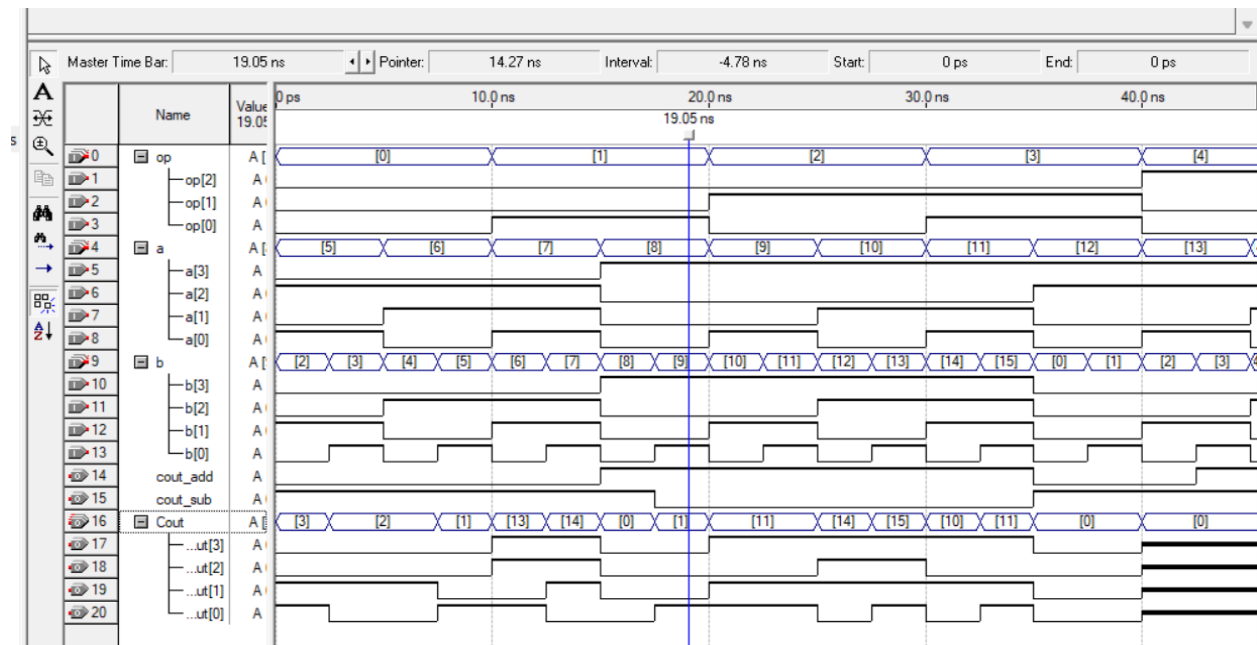
- Waveform: (simulation result):

As shown if the input op = 0 then subtraction, a =5, b=2, then Cout =3.

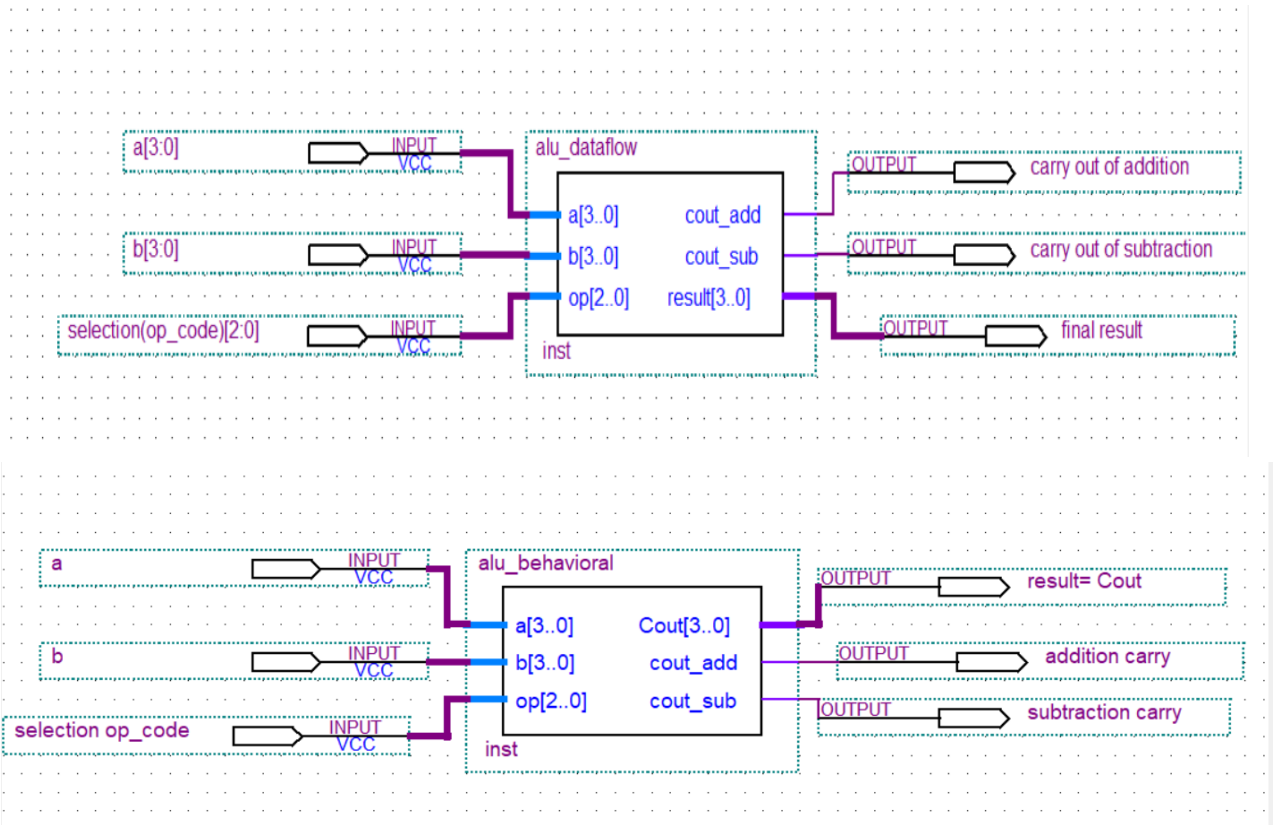
Another case, op = 1 then addition, a = 7, b = 6, then Cout=13.

op = 2 then or, a= 9, b = 11, then Cout =11.

op = 3 then and, a = 11, b =14, then Cout =10.



- **Block Diagram for the ALU :**



- **Conclusion:**

This report outlines the creation of a 4-bit ALU using Verilog HDL. The ALU supports addition, subtraction, and bitwise operations (AND, OR). Each function was built as a separate module using structural modeling, with dataflow connecting them and behavioral modeling managing the top level.

Testing confirmed that the ALU and its components worked as expected. Key outputs like results and carry-out were verified through various test cases.

The project achieved its goals, highlighting the ALU's functionality and modular design while applying core digital design concepts.

تم بحمد الله..